

Python

```
import os
import requests
import dns.resolver
import logging
from datetime import datetime

#
=====
=====
# CONFIDENTIALITY & COMPLIANCE NOTICE:
# This is a sanitized framework. All proprietary algorithms, private API keys,
# and client data have been removed. Strict "no-touch" policies are enforced
# for all public metadata searches.
#
=====
=====

class QuantumOutboundEngine:
    def __init__(self):
        # Enforcing Separation of Concerns: API keys are NEVER hardcoded.
        # They must be injected securely via the host environment.
        self.crm_api_url = os.environ.get("CRM_API_ENDPOINT",
"https://mock-crm.example.com/api/v1")
        self.crm_api_key = os.environ.get("CRM_API_KEY", "REDACTED_FOR_COMPLIANCE")

        logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
        self.logger = logging.getLogger("OutboundEngine")

    def perform_no_touch_discovery(self, target_domain):
        """
        Executes a passive, non-invasive sweep of public metadata and infrastructure.
        """
        self.logger.info(f"Initiating no-touch metadata discovery for: {target_domain}")
        discovery_data = {
            "domain": target_domain,
            "timestamp": datetime.utcnow().isoformat(),
            "infrastructure_signals": [],
            "security_posture_score": 0
        }

        try:
            # IT Infrastructure Mastery: Checking public DNS records (MX, TXT)
            # Identifies email infrastructure (e.g., Google Workspace vs O365)
            mx_records = dns.resolver.resolve(target_domain, 'MX')
```

```

discovery_data["infrastructure_signals"].append(
    f"MX Provider: {str(mx_records[0].exchange)}"
)

# Simple, non-intrusive HTTP header check for security compliance
response = requests.get(f"https://{target_domain}", timeout=5)
headers = response.headers

if 'Strict-Transport-Security' in headers:
    discovery_data["security_posture_score"] += 10
else:
    discovery_data["infrastructure_signals"].append("Missing HSTS - Potential Security
Friction")

except Exception as e:
    self.logger.warning(f"Discovery limited for {target_domain}: {str(e)}")

return discovery_data

def apply_predictive_scoring(self, discovery_data):
    """
    Processes infrastructure signals through an AI-assistive lead scoring logic.
    Focuses on BANT/CHAMP qualification metrics.
    """
    self.logger.info("Applying predictive behavioral analysis and lead scoring.")
    base_score = 50

    # Example Logic: If infrastructure lacks security protocols, it represents
    # a high-value opportunity for technical consulting and remediation.
    signals = discovery_data.get("infrastructure_signals", [])
    if any("Missing HSTS" in signal for signal in signals):
        base_score += 25 # High pipeline friction = High expansion opportunity

    discovery_data["expansion_opportunity_score"] = base_score
    return discovery_data

def sync_to_secure_crm(self, scored_data):
    """
    Pushes the qualified metadata to the CRM (HubSpot/Salesforce) to automate
    touchpoint frequency and manage the high-velocity transaction pipeline.
    """
    if scored_data.get("expansion_opportunity_score", 0) >= 70:
        self.logger.info("High-value target identified. Initiating secure CRM sync.")

        headers = {
            "Authorization": f"Bearer {self.crm_api_key}",
            "Content-Type": "application/json"
        }

```

```
# Simulated API Call
# response = requests.post(f"{self.crm_api_url}/leads", json=scored_data,
headers=headers)
# return response.status_code == 201

self.logger.info("Sync complete. Automated follow-up sequence triggered.")
return True
else:
self.logger.info("Target does not meet high-velocity criteria. Queued for standard
review.")
return False

if __name__ == "__main__":
# Test execution of the framework
engine = QuantumOutboundEngine()
target = "example.com" # Sanitized target

metadata = engine.perform_no_touch_discovery(target)
scored_lead = engine.apply_predictive_scoring(metadata)
engine.sync_to_secure_crm(scored_lead)
```